

Reporte al equipo IA · Podcast del Aula Virtual

Fecha: 2026-09-21 (actualizado 2026-09-23, dos veces el mismo día) • Curso probado: Policía Local de Galicia · temas 1, 3 y 10

Endpoints afectados: POST <https://ia.opox.ai/v1/classroom/podcast> (generación) y GET <https://ia.opox.ai/v1/classroom/podcast/{file}> (audio servido)

Origen del reporte: pruebas en dispositivo real del módulo Podcast + panel de debug instrumentado en el reproductor.

Resumen

Estado actualizado tras confirmación en dispositivo real (2026-09-23):

- **Bug 1 — Seek roto: CAUSA RAÍZ CONFIRMADA en dispositivo real.** El endpoint ya soporta HTTP Range (206 correcto). Pero el mp3 que genera el Motor no trae cabecera `Xing/VBRI` — confirmamos con un panel de debug en el reproductor (ExoPlayer/Android) que el player reporta `duration=0` en vivo, y que cada `seekTo` (adelante o atrás) se resetea a `0.0` sin importar la posición pedida. **De nuestro lado ya implementamos un workaround** (re-codificación a CBR con `ffmpeg` antes de servir el archivo, ver sección dedicada) que debería desbloquear el seek para los usuarios ya mismo, pero seguimos recomendando el fix en el origen (su encoder).
- **Bug 2 — Duración real vs. pedida: NO resuelto, desvío inestable.** La medición de la mañana del mismo día (0.9% de desvío) resultó ser un caso puntual. En pruebas posteriores, incluyendo mediciones de un usuario en dispositivo real, vimos desvíos de -3.3%, -7.7% y -11.7% para el mismo tema y tier en menos de 12 horas. Ver tabla actualizada abajo.

De nuestro lado ya desplegamos el proxy reenviando `Range` y el método HTTP real, y ya usamos la duración real que ustedes devuelven (`duration_seconds`) en vez de un estimado fijo. Ninguno de los dos bugs que quedan es atribuible a nuestro código — ambos apuntan al pipeline de generación del Motor (encoding del mp3 y precisión del guion/TTS respectivamente).

Bug 1 — El usuario no puede adelantar ni retroceder el audio

Cuando el usuario abre un podcast generado y pulsa “15s atrás / 15s adelante”, no pasa nada, o el audio se reinicia desde el principio. Tampoco puede saltar tocando la barra de progreso.

Causa original (servidor)

El endpoint GET [/v1/classroom/podcast/{file}](https://ia.opox.ai/v1/classroom/podcast/{file}) respondía `405 Method Not Allowed` a peticiones HEAD y no exponía `Accept-Ranges/Content-Length`. Ya lo corrigieron (o lo corregimos de nuestro lado con el proxy) — verificado con `curl` en producción, `206 Partial Content` funciona bien.

Bug 1b — falta cabecera Xing/VBRI en el mp3 (encontrado 2026-09-23)

Con el soporte de Range ya funcionando en el servidor, el seek **segua sin funcionar** en dispositivo (Android). Descargamos los primeros 8 KB de un mp3 generado y confirmamos que **no contiene ninguna de las cabeceras Xing, Info ni VBRI** — el marcador que los decoders usan para saber, en un archivo de

bitrate variable (VBR), a qué byte exacto corresponde cada segundo de audio.

Confirmación en dispositivo real (panel de debug + logs del reproductor)

Instrumentamos el reproductor ([expo-audio](#)/ExoPlayer en Android) con un panel de debug que expone su estado interno en vivo. Con un episodio ya reproduciéndose (mostrando “4:39” en la UI, calculado a partir del `duration_seconds` que manda el Motor por otra vía — no del análisis real del archivo por parte del decoder), el estado interno del player mostró:

```
playbackState=ready · timeControlStatus=playing
duration=0 · isLoading=true

17:51:29 skipBy(15) - currentTime=7.5 → pido 22.5
17:51:29 skipBy OK - currentTime ahora=0.0
17:51:33 skipBy(15) - currentTime=4.0 → pido 19.0
17:51:33 skipBy OK - currentTime ahora=0.0
17:51:36 skipBy(15) - currentTime=2.2 → pido 17.2
17:51:36 skipBy OK - currentTime ahora=0.0
17:51:38 skipBy(-15) - currentTime=1.6 → pido 0.0
17:51:38 skipBy OK - currentTime ahora=0.0
```

`duration=0` **reportado por el propio ExoPlayer** — el decoder no logra calcular la duración real del archivo (confirma la falta de cabecera Xing/VBRI, ya no es solo una sospecha por inspección de bytes). Cada llamada a `seekTo/skipBy` responde OK (no lanza excepción) pero el `currentTime` siempre queda en `0.0` sin importar la dirección o posición pedida — comportamiento típico de un player que no puede posicionar un seek dentro de una duración desconocida.

Workaround aplicado de nuestro lado (2026-09-23)

Para no depender del tiempo que les tome corregir el encoder, implementamos un workaround en nuestro backend: la primera vez que se pide un archivo de podcast, lo descargamos completo del Motor y lo re-codificamos con `ffmpeg/libmp3lame` a **CBR (bitrate constante) con cabecera Xing explícita**, y cacheamos el resultado en disco. Las peticiones siguientes (incluidas las de Range que dispara cada seek) se sirven desde ese archivo ya arreglado.

Esto debería desbloquear el seek para los usuarios sin esperar un cambio de su lado. Pero preferimos que el fix llegue también al origen: nuestro workaround agrega latencia extra la primera vez que se reproduce cada episodio (hay que descargar + re-codificar antes de poder responder) y una pérdida de calidad de generación adicional por la re-codificación — aceptable para voz hablada, pero evitable si el mp3 sale bien formado desde el Motor.

Lo que seguimos necesitando de su lado

Que el mp3 se codifique con la cabecera Xing/VBRI incluida (la mayoría de encoders, incluyendo LAME, la agregan automáticamente si no se desactiva explícitamente). Alternativa más simple si el pipeline ya genera CBR: declararlo explícitamente — con CBR los decoders pueden estimar la posición de seek por bitrate fijo sin necesitar la cabecera.

Bug 2 — La duración real del audio no coincide con la pedida

En la app el usuario elige entre “Corta (~5 min)” y “Media (~10 min)”. Enviamos ese valor tal cual en el campo `duracion` del `PodcastIn` ('corta' o 'media').

Fecha / origen	duracion	Target	Duración real	Desvío
2026-09-21 (histórico)	corta	~5 min	8:35	+72%
2026-09-21 (histórico)	media	~10 min	16:07	+61%
2026-09-23 mañana	corta	300 s	303.84 s	+0.9%
2026-09-23 tarde (usuario, disp. real)	corta	300 s	290 s (4:50)	-3.3%
2026-09-23 tarde (nuestra verificación directa)	corta	300 s	276.84 s	-7.7%
2026-09-23 tarde (usuario, disp. real)	corta	300 s	265 s (4:25)	-11.7%

En menos de 12 horas vimos desvíos de +0.9%, -3.3%, -7.7% y -11.7% para el mismo tema y el mismo tier — **el problema no está resuelto, es inestable**. Nuestra propia verificación directa contra `POST /v1/classroom/podcast` (tema 1, curso `ef7d941bea5f41d7`) devolvió:

```
{"resultado": {"duration_seconds": 276.84, "target_seconds": 300, "words": 672}, "tokens_out": 883}
```

Sospechamos que sigue siendo el guion generado por el LLM: con `words`: 672 en esa corrida, y un TTS que ronda ~150 wpm, 672 palabras rinden ~4:29 — coherente con los 276.84 s medidos. El guion mismo ya viene corto/largo de forma variable; no parece un problema de la síntesis de voz en sí. Sugerimos fijar un rango de palabras objetivo más estricto en el prompt del guion, o un paso de validación que regenere si el conteo de palabras se sale de $\pm 5\%$ del target.

De nuestro lado ya usamos la duración real que ustedes devuelven (`duration_seconds`) en vez de un estimado fijo — eso ya está bien de nuestro lado; lo que falta corregir es la precisión del guion/generación en el Motor.

Cómo reproducir

- Generar un podcast desde la app (Aula Virtual → Podcast) con `duracion`: 'corta' o 'media', cualquier tema del curso Policía Local de Galicia.
- Medir la duración real del mp3 devuelto vs. el target — repetir varias veces con el mismo tema, el desvío varía corrida a corrida.
- Sobre el mp3 ORIGINAL del Motor (sin pasar por nuestro workaround de re-codificación): descargar los primeros 2-8 KB y buscar las cadenas `Xing`, `Info` o `VBRI` — no están.

Contacto

Cualquier cosa que necesiten de nuestro lado (payloads exactos, logs, más mediciones) la pasamos sin problema.